

알약 객체 탐지 프로젝트 팀 협업 가이드

역할, 폴더 구조, 파이프라인 경계, Pull Request 절차, 그리고 AI 코딩 도구 사용
규칙

읽는 순서

01 규칙과 역할

대원칙 세 줄 · 다섯 역할 · 파이프라인 지도

02 파이프라인별 경계

하는 일 · 하지 않는 일 · 반려 사례

03 구조와 계약

폴더 스키마 · run(config) · 소유권

04 작업 방식

브랜치 · PR 템플릿 · AI 도구 프롬프트

05 참고

첫날 온보딩 · 사고 대처 · 용어집

SECTION 01

규칙과 역할

규칙이 헛갈릴 때 돌아올 세 문장, 그리고 다섯 사람의 담당 구역.

세 가지 대원칙

01 내 폴더 안에서만 일한다

담당 폴더 하나가 곧 내 작업 범위. 다른 폴더는 읽기만 하고, 수정이 필요하면 이슈로 요청합니다.

02 파이프라인은 서로를 모른다

각 파이프라인은 `run(config)` 하나만 밖으로 내놓습니다. 실행 순서는 메인 파이프라인만 알고 있습니다.

03 모든 변경은 Pull Request로 들어온다

main에 직접 커밋하지 않습니다. 팀장이 경계 침범과 산출물 형식을 확인한 뒤에만 머지됩니다.

역할과 담당 구역

역할	한 문장 책임	담당 폴더
데이터 담당	학습에 들어갈 데이터를 하나의 형식으로 확정한다	pipelines/data/
학습 담당	확정된 데이터로 모델을 학습시켜 체크포인트를 만든다	pipelines/train/
평가 담당	체크포인트를 채점하고 어디서 왜 틀렸는지 분류한다	pipelines/evaluate/
실험기록 담당	흩어진 산출물을 실험 단위로 묶어 API로 내보낸다	pipelines/registry/
팀장	GUI를 만들고, 다섯 파이프라인을 하나로 연결한다	pipelines/web/ · main_pipeline.py

담당자 이름은 [.github/CODEOWNERS](#) 에서 GitHub 계정으로 관리합니다. 사람이 바뀌면 그 한 줄만 고칩니다.

파이프라인 지도



src/main_pipeline.py — 순서를 아는 유일한 파일

팀장 전용 · 파이프라인끼리는 서로를 호출하지 않는다

P5 · 팀장
프론트엔드 GUI
pipelines/web/

화면이 데이터를 보는 유일한 통로
REST API → 실험기록 파이프라인. 화면은 **artifacts/** 폴더를 직접 읽지 않습니다.

SECTION 02

파이프라인별 경계

‘하지 않는 일’ 목록이 곧 침범 금지 구역입니다.

데이터 파이프라인

입력

data/raw/ · config.data

산출물

artifacts/dataset/ – images · labels · splits · dataset_report.json

하는 일

- 어노테이션 형식을 YOLO 하나로 변환
- 손상 이미지 · 중복 · 라벨 오류 제거
- 비율 유지 리사이즈 · EXIF 회전 보정
- 클래스 비율을 지킨 계층 분할 (seed 고정)
- EDA 리포트를 JSON으로 생성

하지 않는 일

- 증강 적용 – 증강은 학습 시점의 일
- 모델 · 학습 코드 수정
- mAP 계산이나 성능 판정
- 분할 규칙 임의 변경
- artifacts/runs/ 폴더 쓰기

학습 파이프라인

입력

artifacts/dataset/ (읽기 전용) ·
config.train

산출물

runs/<exp>/ - weights/best.pt · train_metrics.json ·
config.snapshot.yaml

하는 일

- 사전학습 가중치 로드와 3단계 파인튜닝
- config에 적힌 증강 조합 적용
- epoch별 손실 · 지표를 TensorBoard에 기록
- 검증 성능 최고 시점 체크포인트 저장
- 실행에 쓴 config를 스냅샷으로 남김

하지 않는 일

- 데이터 파일 수정 · 재분할 · 라벨 손질
- 최종 mAP 판정과 실패 이미지 생성
- submission.csv 생성
- 실험 목록 · 색인 갱신
- GUI나 API 코드 호출

평가 · 오류분석 파이프라인

입력

runs/<exp>/weights/best.pt · 검

증 분할

산출물

eval/ - metrics.json · per_class_ap.json · confusion_matrix.json ·

failures/ · submission/

하는 일

- mAP@0.5 · mAP@0.5:0.95 · Precision · Recall
- 클래스별 AP와 혼동행렬 생성
- 실패 이미지를 4개 유형으로 분류해 저장
- 추론 임계값 스위치 (conf · NMS)
- 제출용 submission.csv 생성

하지 않는 일

- 재학습 실행 · 가중치 파일 수정
- 학습 파라미터를 바꿔 보기
- 데이터 분할 변경
- API 응답 형식 · DB 스키마 수정
- Kaggle 점수를 손으로 써넣기

실험기록 · API 파이프라인

입력

artifacts/ 전체 (읽기 전용) · GUI 등록 요청

산출물

registry/index.json · experiments.sqlite · REST API

하는 일

- 실험 ID 발급과 설정 파일 생성
- runs/ 폴더를 스캔해 실험 색인 갱신
- 지표 · 곡선 · 실패 이미지 경로를 API로 제공
- Kaggle 점수를 실험 ID에 연결해 저장
- 산출물 규격 검증 · 어긋나면 거부

하지 않는 일

- 지표 재계산 — 평가의 숫자를 그대로 전달
- 학습이나 추론 실행
- 이미지 생성 · 가공
- 프론트엔드 화면 코드 작성
- 남의 산출물 파일 수정 · 삭제

프론트엔드와 메인 파이프라인

입력

산출물

실험 기록 API 응답만

설정 GUI · 실험 비교 · 학습 곡선 · 실패 갤러리 · 제출 파일 내려받기

하는 일

- 설정 화면에서 실험 등록 요청 전송
- 학습 파라미터와 추론 임계값을 구분해 표시
- 두 실험을 나란히 비교
- 메인 파이프라인 조립과 실행 순서 관리

하지 않는 일

- 모델 학습 · 추론 연산
- 지표를 화면에서 계산하거나 보정
- artifacts/ 폴더를 직접 읽고 쓰기
- 남의 파이프라인 내부 코드 수정

직교성 규칙 세 줄

R1 다른 파이프라인 폴더를 **import** 하지 않는다. 필요한 값은 config나 산출물 파일로 받습니다.

R2 다른 파이프라인이 만든 파일을 수정하지 않는다. 읽기만 합니다.

R3 파이프라인끼리 **직접 부르지 않는다**. 실행 순서는 메인 파이프라인만 압니다.

예외는 두 곳뿐

`src/common/` 과 `contracts/` 는 누구나 import 할 수 있지만, 수정은 팀장만 합니다.

이런 PR은 반려됩니다

변경 내용	올바른 방법
<code>data/</code> 에서 <code>train.augment</code> 를 import	증강은 학습의 일. config 항목으로 요청합니다
<code>train/</code> 이 <code>dataset/labels/*.txt</code> 수정	라벨 오류는 이슈로. 데이터 담당이 고칩니다
<code>evaluate/</code> 가 학습을 다시 실행	평가는 채점만. 재학습은 새 실험으로 등록
<code>registry/</code> 가 mAP 를 다시 계산	숫자의 출처는 한 곳이어야 합니다
<code>web/</code> 이 <code>artifacts/</code> 를 직접 읽음	화면은 API만 통해 데이터를 봅니다
<code>requirements.txt</code> · <code>configs/</code> 수정	공용 파일은 팀장만. 이슈로 요청합니다
<code>run(config)</code> 반환 키 이름 변경	계약 변경은 팀장 승인이 먼저입니다

SECTION 03

구조와 계약

저장소 하나, 파이프라인 폴더 다섯, 연결은 파일 하나.

폴더 스키마

```

pill-detection/
├── README.md                팀장
├── CLAUDE.md · AGENTS.md   팀장
├── requirements.txt         팀장
├── .github/
│   ├── CODEOWNERS
│   └── pull_request_template.md
├── configs/                 팀장
│   ├── base.yaml
│   ├── env.local.yaml · env.colab.yaml
│   └── experiments/
├── contracts/               팀장
│   ├── artifacts.md
│   └── schemas/
└── data/raw/                git 제외
  
```

```

artifacts/                  git 제외
├── dataset/                 쓰기: 데이터
├── runs/exp_014/            쓰기: 학습 · 평가
└── registry/                쓰기: 실험기록

src/
├── main_pipeline.py         팀장
├── common/                  팀장
└── pipelines/
    ├── data/                데이터 담당
    ├── train/               학습 담당
    ├── evaluate/            평가 담당
    ├── registry/            실험기록 담당
    └── web/                  팀장
  
```

오른쪽 주석이 그 경로를 수정할 수 있는 사람입니다. 그 외에는 읽기만 합니다.

폴더 내부는 다섯 개가 모두 같은 모양

```
src/pipelines/train/  
├── CLAUDE.md      폴더 규칙  
├── AGENTS.md      같은 내용  
├── __init__.py  
├── run.py          ← 유일한 입구  
├── finetune.py     내부 구현  
├── augment.py      내부 구현  
└── tests/test_run.py
```

```
def run(config: dict) -> dict:  
    """자기 소유 폴더에만 파일을 쓴다"""  
    return {  
        "status":      "ok",  
        "artifacts": {"dataset_dir": "..."},  
        "summary":     {"images": 4120},  
        "message":     "",  
    }
```

밖에서 `finetune.py` 를 import 하는 코드가 보이면, 그 순간 두 파이프라인이 붙어 버린 것입니다.

메인 파이프라인이 하는 일

```
# src/main_pipeline.py - 팀장만 수정
STAGES = [("data", data), ("train", train), ("evaluate", evaluate), ("registry", registry)]

def main(config_path, only=None):
    config = load_config(config_path)      # base + env + experiment 병합
    results = {}
    for name, module in STAGES:
        if only and name not in only:      # 담당자는 자기 단계만 돌린다
            continue
        out = module.run(**config, "inputs": results)
        if out["status"] != "ok":
            raise SystemExit(f"[{name}] 중단: {out['message']}")
        results[name] = out["artifacts"]    # 다음 단계로 경로만 넘긴다
```

자기 단계만 실행

```
python -m src.main_pipeline --config configs/experiments/exp_000_baseline.yaml --only train
```

파일 소유권과 CODEOWNERS

경로	수정	나머지
pipelines/<내 폴더>/	담당자	열지 않음
artifacts/<내 산출물>/	담당자	읽기만
common/ · contracts/	팀장	import 허용
configs/ · requirements	팀장	이슈로 요청
main_pipeline.py	팀장	수정 금지

.github/CODEOWNERS

```
*                                @teamlead
/src/pipelines/data/            @data-owner
/src/pipelines/train/           @train-owner
/src/pipelines/evaluate/        @eval-owner
/src/pipelines/registry/        @reg-owner
/src/main_pipeline.py           @teamlead
/configs/ /contracts/           @teamlead
```

저장소 설정 · 팀장이 1회

- Require a pull request before merging
- Require approvals: 1 · Code Owners 리뷰 필수
- Force push · 삭제 허용 끄기

SECTION 04

작업 방식

브랜치 · Pull Request · AI 코딩 도구 프롬프트.

브랜치 이름과 PR 템플릿

브랜치 이름 규칙

```
pipeline/<코드>/<작업 요약>
```

```
pipeline/data/coco-to-yolo
```

```
pipeline/train/exp014-lr5e4
```

```
pipeline/evaluate/failure-gallery
```

```
pipeline/registry/kaggle-score-api
```

```
pipeline/web/compare-view
```

코드는 폴더 이름과 같습니다: data · train · evaluate · registry · web. 한 브랜치에는 한 가지 작업만 담습니다.

.github/pull_request_template.md

```
## 무엇을 바꿨나
```

```
## 왜 바꿨나
```

```
## 검증 방법
```

```
실행한 명령 / 결과 / before-after
```

```
## 자기 점검
```

- [] 내 폴더 밖의 파일을 바꾸지 않았다
- [] 다른 파이프라인을 `import` 하지 않았다
- [] `run(config)` 반환 키를 유지했다
- [] 산출물을 내 폴더에만 썼다
- [] 한 번에 한 가지만 바꿨다

팀장 머지 전 체크리스트

① 경계 침범

변경 파일이 담당 폴더 안에만 있는지 확인합니다. 출력이 한 줄이라도 있으면 이유를 묻고, 설명이 없으면 반려합니다.

```
git diff --name-only main... | grep -v "^src/pipelines/train/"
```

② 산출물 스키마

반환 키 네 개가 그대로인지, 산출물 파일 이름이 `contracts/artifacts.md` 와 맞는지 확인합니다. 새 산출물이 생겼다면 규격 문서도 같은 PR에 있어야 합니다.

③ 재현성

seed 고정 · 절대경로 없음 · 실행 명령이 PR 본문에 있음 · 지표가 바뀌는 변경인데 before / after가 없으면 반려합니다.

오염이 의심되면 머지하지 말고 `git checkout main -- <침범한 경로>` 로 그 파일만 원복하게 합니다. PR은 유지되고 침범만 사라집니다.

AI 코딩 도구 사용 규칙

폴더마다 규칙 파일 • 두 파일은 내용 동일

```
# src/pipelines/train/CLAUDE.md
#   (AGENTS.md 도 같은 내용)

# 이 폴더: 학습 파이프라인
- 담당: 학습 담당
- 입구: run(config) 하나뿐.
      시그니처를 바꾸지 않는다.
- 읽기: dataset/ • configs/ • common/
      (모두 수정 금지)
- 쓰기: runs/<exp_id>/ 와 이 폴더 안
- 금지: 다른 pipelines/* import,
      main_pipeline.py 수정
- 막히면: 스스로 다른 폴더를 고치지
      말고 사용자에게 보고한다
```

도구를 쓸 때의 습관 네 가지

- 01 “저장소 전체를 정리해 줘” 같은 요청은 하지 않습니다. 범위 없는 지시가 경계를 넘게 만듭니다.
- 02 한 번에 파일 세 개 이하만 바꾸게 합니다. 커지면 작업을 쪼갭니다.
- 03 끝났다고 하면 반드시 **git status** 를 눈으로 확인합니다.
- 04 이해하지 못한 코드로 PR을 올리지 않습니다. 리뷰에서 설명을 요구받습니다.

복붙용 시작 프롬프트

① 작업 시작

너는 이 저장소의
"학습 파이프라인" 담당이다.

범위 : `src/pipelines/train/`
안의 파일만 수정한다.

금지 : 다른 `pipelines/*`,
`main_pipeline.py`, `configs/`,
`requirements` 수정.
다른 파이프라인 `import`.

계약 : `run(config)` 반환 형태를
바꾸지 않는다.

할 일: (한 가지만 적는다)

끝나면 변경 파일 목록과
`git status` 를 보여 줘라.

② 막혔을 때

증상 :
(에러 메시지를 그대로)

실행한 명령 :
(명령을 그대로)

원인을 먼저 한 문장으로
설명한 뒤 수정안을 보여 줘라.

수정은 `src/pipelines/train/`
안에서만 한다.

원인이 다른 폴더에 있으면
고치지 말고, 어느 파일의
무엇이 문제인지만 알려 줘라.

③ PR 올리기 전

아래 세 가지를 점검해서
결과만 알려 줘라.

1. `git diff --name-only main...`
에서 `train/` 밖의 파일이
있으면 목록으로.

2. `train/` 안에서 다른
파이프라인을 `import` 한 줄을
찾아서.

3. `run(config)` 반환 키가
`contracts/schemas/` 와
맞는지.

문제가 없으면 PR 본문을
템플릿 형식으로 작성해 줘라.

실행 환경과 경로 규칙

로컬 GPU

소형 모델 탐색 실험, 파이프라인 오류 잡기
batch를 낮췄다면 config에 그 값을 남깁니다

Colab

중형 이상 모델의 본 학습, epoch가 긴 실험
세션이 끊깁니다. 체크포인트를 Drive에 복사

Kaggle

최종 검증과 제출
제출 횟수 제한. 로컬이 개선된 실험만 올립니다

```
# configs/env.colab.yaml
paths:
  raw_data: /content/drive/.../raw
  artifacts: /content/drive/.../artifacts

# 실행할 때 환경만 바꿔 끼운다
--env colab
```

경로 규칙

- 코드 안에 절대경로를 쓰지 않습니다 — 반려 대상
- 경로는 항상 `common/paths.py` 를 통해 얻습니다
- config 스냅샷 · seed · 분할 파일이 같으면 재현됩니다

실험 ID와 파일 이름 규칙

대상	형식	예시
실험 ID	exp_NNN	exp_014
설정 파일	exp_NNN_<바꾼 것>.yaml	exp_014_lr5e4.yaml
체크포인트	best_epoch<N>.pt	best_epoch62.pt
제출 파일	sub_NNN_expNNN.csv	sub_007_exp014.csv
브랜치	pipeline/<코드>/expNNN-<요약>	pipeline/train/exp014-lr

- 실험 번호는 GUI에서 발급받습니다. 사람이 임의로 정하지 않습니다.
- 번호는 재사용하지 않습니다. 실패한 실험도 번호를 남겨 둡니다.
- 학습 파라미터 변경과 추론 임계값 변경은 다른 실험으로 기록합니다.

SECTION 05

참고

첫날 온보딩 · 사고 대처 · 용어집 · 담당자 카드.

첫날 온보딩 절차

목표는 학습 성공이 아니라, 아주 작은 PR 하나를 머지까지 통과시키는 것입니다.

01 저장소를 clone 하고 담당 폴더 이름을 확인

02 Python 3.11 가상환경 + requirements 설치

03 샘플 데이터를 data/raw/ 에 두기 (git 제외 폴더)

04 자기 단계만 실행 — `--only data`

05 담당 폴더의 CLAUDE.md 와 contracts/artifacts.md 읽기

06 프롬프트 ①을 붙여넣고, 할 일에는 ‘주석 한 줄 추가’만 적기

07 브랜치를 만들어 push하고 PR을 열기 · 템플릿 끝까지 채우기

08 팀장 리뷰를 받고 머지 확인 — 여기까지가 첫날 완료

자주 나는 사고와 대처법

증상	원인	대처
내 컴퓨터에서만 돌아간다	절대경로 하드코딩	경로를 env yml로 옮기고 paths 유틸을 씁니다
AI가 남의 폴더를 고쳤다	범위 없는 지시	해당 경로만 원복하고 프롬프트 ①로 다시 시작
merge conflict가 났다	경계를 넘은 신호	내 폴더만 건드렸다면 충돌은 나지 않습니다. 팀장에게 알립니다
점수는 올랐는데 이유를 모른다	한 번에 여러 개 변경	되돌려서 하나씩 다시. config diff를 PR에 첨부
라벨이 틀린 것 같다	어노테이션 오류	직접 고치지 않고 이슈로. 데이터 담당만 수정
Colab 세션이 끊겼다	런타임 제한	체크포인트를 Drive에 주기 저장하고 이어서 학습
패키지 버전이 충돌한다	각자 설치	직접 requirements를 고치지 않고 이슈로 요청
리더보드만 조금씩 오른다	리더보드 과적합	로컬 검증이 함께 오르지 않은 설정은 채택하지 않습니다

용어집 ① 과제와 지표

객체 탐지

무엇이 어디에 몇 개 있는지까지 답하는 일. 분류는 '무엇'만 답합니다

바운딩 박스

알약 하나를 둘러싼 네모. 좌표 네 개로 표현합니다

IoU

예측 박스와 정답 박스가 겹친 넓이 ÷ 합친 넓이

conf · NMS 임계값

학습된 모델의 답을 어디서 자를지 정하는 값. 재학습 없이 바꿉니다

Precision

찾았다고 한 것 중 맞은 비율. 낮으면 없는 알약을 자꾸 찾습니다

Recall

있는 것 중 찾아낸 비율. 낮으면 있는 알약을 놓칩니다

mAP@0.5

IoU 0.5 기준 하나로 켄 평균 성적. 클래스 구분 능력을 봅니다

mAP@0.5:0.95

기준을 0.5에서 0.95까지 올려 가며 평균 낸 값. 박스 정확도까지 봅니다

용어집 ② 학습과 기록

전이학습 · 파인튜닝

남이 학습해 둔 모델을 가져오는 것이 전이학습, 우리 데이터로 이어서 학습하는 것이 파인튜닝

증강

학습 이미지를 뒤집고 밝기를 바꿔 가며 넣는 것. 모델의 실력을 바꿉니다

TTA

예측할 때 같은 이미지를 여러 변형으로 넣어 합치는 것. 모델은 그대로입니다

리더보드 과적합

제출 점수만 보고 설정을 고치다 모델이 아니라 시험지에 맞춰지는 것

체크포인트

학습 도중 저장한 모델 파일. 검증 성능이 가장 좋았던 시점을 씁니다

계층 분할 · seed

클래스 비율을 지켜 나누는 방식과, 그 나눔을 재현하기 위한 고정 숫자

과적합

학습 데이터만 잘 맞히게 되는 현상. 학습 손실은 내려가는데 검증 성능이 멈춥니다

산출물 · artifact

파이프라인이 만들어 다음 단계에 넘기는 파일. 우리 팀의 유일한 소통 수단입니다

담당자 요약 카드

P1

데이터 담당

내 산출물

artifacts/dataset/

손대면 안 되는 곳

train · evaluate · registry · web · configs · runs

P2

학습 담당

내 산출물

runs/<exp>/weights ·
train_metrics.json

손대면 안 되는 곳

dataset 파일 수정 · eval/ · 제출 파일 · configs

P3

평가 담당

내 산출물

runs/<exp>/eval · submission/

손대면 안 되는 곳

weights 수정 · 재학습 · 학습 파라미터 · API 스키
마

P4

실험기록 담당

내 산출물

registry/index.json · REST API

손대면 안 되는 곳

지표 재계산 · 학습 실행 · 남의 산출물 · 화면 코드

P5

팀장

내 산출물

web/ · main_pipeline.py · configs
· contracts

하지 않는 일

남의 파이프라인 내부를 직접 고쳐 주기 — 이슈와
리뷰로

첫 작업 순서

팀장이 빈 run.py 다섯 개와 main_pipeline.py
로 끝까지 도는 껍데기를 먼저 머지합니다. 그 뒤
에 각자 자기 폴더를 채웁니다.

경계가 분명하면 다섯 명이 동시에 일할 수 있습니다

내 폴더 안에서만

남의 폴더는 읽기만. 수정은 이슈로 요청합니다

계약은 함수 하나

오가는 것은 경로 문자열뿐. 순서는 메인만 압니다

모든 변경은 PR로

팀장이 경계와 산출물을 확인한 뒤 머지합니다